

CÉGEP STE-FOY
PROGRAMMATION OBJET 1– 420-W20-SF

Travail Pratique 2

Préparé par
Raphaël Paradis et Emmanuel Garreau

1 Résumé

Dans ce travail pratique, vous n'incarne pas un héros, mais le **répartiteur (Dispatcher)** d'une agence de super-héros. Votre rôle est d'analyser des situations de crise et d'y assigner l'équipe la plus apte à réussir la mission, tout en gérant la fatigue de vos effectifs.

2 Conditions de réalisation

Valeur de la note finale	Type	Remise + Validation
Code - 8 % Validation - 5%	En équipe	Lundi 5 octobre 2026 8h30

3 Spécifications

3.1 Classes à coder

Vous aurez **au minimum** les classes suivantes à produire pour ce travail (en plus des classes de tests pertinentes). Le diagramme de classe **n'est pas fourni** pour ce travail et seul un projet de base contenant *Program.cs*, *RandomGenerator.cs* et quelques données vous est fourni. Il est donc fortement recommandé de réfléchir d'abord par vous-même à votre architecture (et à faire un diagramme sommaire, points bonus), puis d'implémenter votre solution par la suite.

À noter également que le travail est séparé en deux mandats distincts (un par étudiant) ainsi qu'une partie commune. Donc, pour ce travail, l'étudiant 1 va concevoir et rédiger le code de toutes les classes liées au **Héros** et l'étudiant 2 va faire le même travail pour les classes liés au **Événements**. Les deux étudiants vont finalement travailler conjointement pour la réalisation du **Répartiteur (Dispatcher)**, qui constitue en quelque sorte le moteur de jeu. Prenez d'ailleurs bien soin de spécifier en entête de chaque fichier qui avait la responsabilité d'en rédiger le code comme suit :

```
// Étudiant [1 ou 2] : [Votre nom complet]
```

Prenez toutefois le temps de lire les deux mandats! Certaines choses sont assez semblables entre les deux.

Mandat de l'étudiant 1

L'énumération **EventOutcome** (**ResultatEvenement**)

Énumération contenant les valeurs suivantes : *Succes, Echec*

La classe **Event** (**Événement**)

Description des données membres :

- **nom** : Le nom de l'événement. Doit être généré aléatoirement (voir les tableaux dans la classe RandomGenerator).
- **difficulté** : La difficulté de l'événement. Aide à déterminer les statistiques de l'événement.

De plus, la classe Événement doit contenir les statistiques de ce dernier. Chaque événement a quatre données membres qui définissent la situation : vigueur, mobilité, intelligence, charisme. Chaque statistique est représentée par un nom et par un nombre entier qui représente à quel point cet attribut est nécessaire pour résoudre la situation. Ce nombre est toujours plus grand que 0 et n'a pas de limite (contrairement à celles du héros). **Attention! C'est à vous de trouver une solution efficace en suivant les principes de programmation vu en classe pour implémenter ces statistiques.**

Le constructeur d'un Événement recevra les paramètres suivants : son nom et sa difficulté. Les statistiques sont instanciées selon un algorithme basé sur la difficulté (voir une des méthode ci-bas).

Description de **quelques** méthodes – vous devez déduire la visibilité de chacune et il sera probablement nécessaire de *découper votre code en ajoutant des méthodes (publiques et privées)*. Vous devrez déduire les types des paramètres. Vous pouvez traduire les noms en anglais sans problème.

- **float CalculerProbabiliteSucces([...])**
Reçoit la liste de héros qui participe à l'événement en paramètre. On calcule la probabilité de succès d'un événement en faisant le calcul suivant : pour chaque statistique (vigueur, mobilité, intelligence et charisme), on divise : **la somme de la valeur de la statistique de tous les héros participant à l'événement** par : **la valeur de la statistique correspondante de l'événement**. Cela donne la probabilité de succès par statistique (qui ne peut pas dépasser 100%). On fait ensuite la moyenne de ces probabilités, ce qui nous donnera la chance de succès pour l'événement.
Exemple pour une statistique : deux héros tentent de résoudre un événement. Le premier a 5 de vigueur et le second a 2 de vigueur. L'événement a une statistique de vigueur de 8. On obtient donc : $7/8 = 0,875 = 87,5\%$ de chance de réussite pour la vigueur. On recommence ensuite l'opération avec les autres statistiques et on fait la moyenne des pourcentages, ce qui nous donne notre chance finale.
- **ResultatEvenement ResoudreEvenement([...])**
Reçoit la liste de héros qui participe à l'événement en paramètre. Si la liste est vide ou nulle, l'événement est automatiquement un échec. Si la liste contient un ou plusieurs héros, on calcule la probabilité de succès selon cette liste, puis on génère un nombre aléatoire. Si ce nombre est plus petit que la probabilité de réussite, l'événement est résolu avec succès. Dans le cas contraire, l'événement est résolu avec échec. Finalement, on résout l'événement pour tous les héros ayant participé à l'événement (voir les méthodes de Héros) et on retourne le résultat.

- **[?] CreerStatistiquesSelonDifficulte()**

Le but de cette méthode est de distribuer des « points » dans les statistiques de l'événement de manière aléatoire. Le nombre de point à distribuer est défini ainsi : la difficulté de l'événement + 2. Chaque statistique de l'événement est initialement affectée à 1 (le minimum) et on distribue ensuite chaque point entre les quatre statistiques aléatoirement. À vous de voir comment vous y prenez selon votre implémentation!

Mandat de l'étudiant 2

La classe Hero (Héros)

Description des données membres :

- **nom :** Le nom du héros. Doit contenir au maximum 25 caractères.
- **niveau :** Le niveau du personnage. Commence toujours à 1 et ne peut jamais être plus bas que ce nombre. Indicateur de son nombre d'événements réussis.
- **reposRestant :** le nombre d'événements (ou de « tours ») qui doivent se passer avant que le héros puisse retourner sur le terrain. Le héros est prêt à être envoyé lorsque ce nombre est à 0 et il doit se reposer lorsque ce nombre est supérieur à 0 (il ne peut pas être déployé pendant ce temps).
- **historique :** Un dictionnaire contenant le nom de l'événement (clé) ainsi que le résultat de l'intervention : succès ou échec? (valeur).

De plus, la classe Héros doit contenir les statistiques de ce dernier. Chaque héros a quatre données membres qui définissent ses compétences : vigueur, mobilité, intelligence, charisme. Chaque statistique est représentée par un nom et par un nombre entier qui représente à quel point il est compétent dans le domaine. Ce nombre est toujours plus grand que 0 et ne peut pas dépasser 10. **Attention! C'est à vous de trouver une solution efficace en suivant les principes de programmation vu en classe pour implémenter ces statistiques.**

Le constructeur d'un Héros recevra les paramètres suivants : son nom et ses statistiques. Le reste doit tout de même être initialisé.

Description de **quelques** méthodes - vous devez déduire la visibilité de chacune et il sera probablement nécessaire de *découper votre code en ajoutant des méthodes (publiques et privées)*. Vous devrez déduire les types des paramètres. Vous pouvez traduire les noms en anglais sans problème.

- **bool EstEnRepos()**
Retourne vrai si le repos restant est supérieur à 0, faux sinon.
- **void SeReposer()**
Permet de réduire le repos restant (décrémente *reposRestant* de 1).

- **void ResoudreEvenement([...])**

Reçoit l'événement auquel le héros a participé ainsi que le résultat de la mission (succès ou échec) en paramètre. Si la mission est un succès, le héros augmente de niveau (voir méthode suivante). Sinon, le héros est blessé, ce qui augmente son temps de repos d'un événement. Dans tous les cas, le héros tombe en repos : pour deux tours si l'événement était un succès ou pour trois si c'était un échec. Finalement, l'événement est noté dans l'historique du héros (voir donnée membre *historique*).

- **void MonterNiveau()**

Incrémente le niveau du héros de 1 et incrémente une statistique aléatoire du héros.

Mandat commun

La classe **Dispatcher (Repartiteur)**

Description des attributs :

- **nbSucces** : le nombre d'événements résolus avec succès. Sera affiché dans la console.
- **nbEchec** : le nombre d'événements ayant résulté en échec. La partie prend fin lorsqu'on atteint trois échecs.
- **hérosDisponibles** : les héros disponibles au déploiement. Contient tous les héros, même ceux au repos.
- **difficulteGlobale** : la difficulté des événements. Celle-ci commence à 1 et augmente de 1 à chaque fois que deux événements ont été résolus avec succès.

Le constructeur ne prend aucun paramètre. Vous devez toutefois instancier toutes les données membres, y compris votre liste de héros disponible (**qui doit contenir absolument cinq héros**). Voici un tableau contenant mes héros pour vous inspirer (mais vous pouvez créer la vôtre sans problème) :

Nom	Statistiques			
	Mobilité	Vigueur	Intelligence	Charisme
Ultra Raph	1	3	2	2
Supermmanuel	3	1	2	2
PF CaméLéon	1	2	4	1
Aliday	3	2	2	1
GP le PC	2	2	2	2

Description de la méthode publique, soit la boucle de jeu :

- **public void RepartirHeros()**

Cette méthode est une boucle qui s'exécute jusqu'à ce que le répartiteur ait obtenu trois échecs. On commence par afficher les nombres de succès et d'échecs, puis on génère un nouvel événement à gérer avec un nom aléatoire et une difficulté selon la difficulté globale. On affiche ensuite cet événement ainsi que la liste de tous les héros (il devrait être évident quels héros sont disponibles et lesquels sont au repos). L'utilisateur choisit ensuite quels héros il souhaite envoyer pour résoudre le problème. Il ne peut pas envoyer le même héros plusieurs fois ou un héros au repos. La probabilité de succès est affichée et se met à jour selon la liste de héros envoyée. Une fois la liste de héros envoyés décidée, on **résout l'événement**. Si celui-ci est un succès, on augmente le nombre de succès et, sinon, on augmente le nombre d'échecs. Idéalement, l'état de la simulation est toujours représenté dans l'affichage (en d'autres mots, les informations importantes pour l'utilisateur sont affichées au moment opportun. Vous pouvez vous fier aux exemples dans la section suivante pour cela). Finalement, les héros n'ayant pas participé à l'événement se reposent.

Lorsque les trois échecs sont atteints, on affiche l'historique des cinq héros et le programme se termine.

Vous devriez créer d'autres méthodes privées ici afin de bien découper votre code. Des méthodes de saisie (et de validation des saisies) et potentiellement d'affichage seraient pertinentes.

La classe Program

La classe **Program** va seulement contenir le *Main* et y instancier les objets nécessaires pour démarrer la répartition des héros et la gestion des événements.

3.2 Exemples :

Exemple – Gestion de quelques événements :

```
Nombre de succès : 0 , Nombre d'échecs : 0

Situation nécessitant une intervention : Neutraliser le maire, Difficulté : 1
Attributs nécessaires pour répondre à la situation : Vigor : 1  Mobility : 1  Intellect : 3  Charisma : 2
1. [0] Ultra Raph , Level 1 : Vigor : 1  Mobility : 3  Intellect : 2  Charisma : 2
2. [0] Supermanuel , Level 1 : Vigor : 3  Mobility : 1  Intellect : 2  Charisma : 2
3. [0] PF CaméLéon , Level 1 : Vigor : 1  Mobility : 1  Intellect : 4  Charisma : 2
4. [0] Ali le Boxer , Level 1 : Vigor : 3  Mobility : 2  Intellect : 1  Charisma : 2
5. [0] GP le Super-Calculateur , Level 1 : Vigor : 1  Mobility : 1  Intellect : 4  Charisma : 2
Héros envoyés sur la scène : []
Chance de réussite actuelle : 0,00 %
2
Héros envoyés sur la scène : [Supermanuel]
Chance de réussite actuelle : 91,67 %

La situation est sous contrôle!

Appuyez sur une touche pour confirmer et passer à l'événement suivant...
```

L'utilisateur entre les héros qu'il souhaite envoyer pour régler l'événement après la ligne « Chance de réussite actuelle ». Il peut envoyer autant de héros qu'il le souhaite. La saisie de héros se termine lorsque l'utilisateur entre 0 ou fait simplement la touche « Entrée » sans spécifier un numéro. Le but est donc d'envoyer un nombre suffisant de héros pour avoir un haut pourcentage de réussite. À noter toutefois que même si le pourcentage est haut, on peut être malchanceux et échouer tout de même...

```
Nombre de succès : 1 , Nombre d'échecs : 0

Situation nécessitant une intervention : Évacuer Poufi le chien, Difficulté : 1
Attributs nécessaires pour répondre à la situation : Vigor : 3  Mobility : 1  Intellect : 2  Charisma : 1
1. [0] Ultra Raph , Level 1 : Vigor : 1  Mobility : 3  Intellect : 2  Charisma : 2
2. [2] Supermanuel , Level 2 : Vigor : 3  Mobility : 2  Intellect : 2  Charisma : 2
3. [0] PF CaméLéon , Level 1 : Vigor : 1  Mobility : 1  Intellect : 4  Charisma : 2
4. [0] Ali le Boxer , Level 1 : Vigor : 3  Mobility : 2  Intellect : 1  Charisma : 2
5. [0] GP le Super-Calculateur , Level 1 : Vigor : 1  Mobility : 1  Intellect : 4  Charisma : 2
Héros envoyés sur la scène : []
Chance de réussite actuelle : 0,00 %
1
Héros envoyés sur la scène : [Ultra Raph]
Chance de réussite actuelle : 83,33 %
3
Héros envoyés sur la scène : [Ultra Raph, PF CaméLéon]
Chance de réussite actuelle : 91,67 %

La situation est hors de contrôle!

Appuyez sur une touche pour confirmer et passer à l'événement suivant...
```

On ne peut pas entrer un héros qui se repose ou qui n'existe pas. Le programme ne devrait jamais planter.

```
Nombre de succès : 1 , Nombre d'échecs : 1

Situation nécessitant une intervention : Capturer les étudiants perdus, Difficulté : 1
Attributs nécessaires pour répondre à la situation : Vigor : 2  Mobility : 2  Intellect : 2  Charisma : 1
1. [3] Ultra Raph , Level 1 : Vigor : 1  Mobility : 3  Intellect : 2  Charisma : 2
2. [1] Supermanuel , Level 2 : Vigor : 3  Mobility : 2  Intellect : 2  Charisma : 2
3. [3] PF CaméLéon , Level 1 : Vigor : 1  Mobility : 1  Intellect : 4  Charisma : 2
4. [0] Ali le Boxer , Level 1 : Vigor : 3  Mobility : 2  Intellect : 1  Charisma : 2
5. [0] GP le Super-Calculateur , Level 1 : Vigor : 1  Mobility : 1  Intellect : 4  Charisma : 2
Héros envoyés sur la scène : []
Chance de réussite actuelle : 0,00 %
1
Le héros doit se reposer!
Héros envoyés sur la scène : []
Chance de réussite actuelle : 0,00 %
6
Le numéro entré n'est pas valide
|
```

Lorsqu'on atteint trois échecs, on affiche l'historique de chaque héros et on quitte ensuite le programme.

```
Votre quart de travail est terminé! Voici la performance de vos héros :

Ultra Raph : { Neutraliser les étudiants perdus : Success, Protéger la petite famille : Success, Évacuer tante Yvette : Success, Libérer tante Yvette : Failure, }
Supermanuel : { Neutraliser le passager du tramway : Success, Évacuer Grand-Mère : Success, Capturer tante Yvette : Failure, }
PF CaméLéon : { Capturer la petite famille : Success, Neutraliser les étudiants perdus : Success, Neutraliser la petite famille : Success, }
Ali le Boxer : { Retrouver la petite famille : Success, Évacuer Poufi le chien : Failure, Neutraliser Poufi le chien : Success, }
GP le Super-Calculateur : aucun événement résolu
```

Vous n'avez pas à suivre l'affichage à la lettre, mais assurez-vous de bien représenter ce qui se passe (et n'hésitez pas l'améliorer également)!

4 Tests unitaires

Les tests unitaires sont à votre charge. Déterminez ce qui est utile et possible à tester et ce qui ne l'est pas.

5 Évaluation

Vous trouverez dans un fichier sur LÉA (donné ultérieurement) la grille d'évaluation utilisée pour la correction de ce travail pratique.

À noter que la codification (qualité du code) est évaluée **en équipe**. Il est donc fortement recommandé de faire des revues de code en équipe pour commenter la qualité du code de votre coéquipier et s'assurer que le code soit bon dans son ensemble.

Vous serez fortement pénalisés dans les cas suivants :

1. Si votre code ne compile pas ou qu'il contient des erreurs majeures empêchant d'utiliser l'application correctement, vous serez pénalisés d'au moins 15% de la note finale (en plus des points perdus sur les fonctionnalités ne pouvant pas être évaluées à cause de ces erreurs).
2. Dans le cas où quelqu'un n'aurait accompli que quelques fonctionnalités dans toutes celles demandées (et que la quantité de code est donc moindre par rapport à ce qui est attendu), la proportion des points attribués à la codification (qualité du code) ne pourra pas dépasser la proportion des points atteinte pour les fonctionnalités. Par exemple, si une personne cumule 30 points sur les 60 possibles pour les fonctionnalités ($30/60 = 50\%$), elle ne pourra pas avoir plus de 20 points pour la qualité du code (soit 50% des 40 points possibles). La note attribuée à la codification sera alors proportionnelle à ce rapport (par exemple, quelqu'un qui aurait eu 80% dans la codification pour ce travail, soit 32 points sur 40, aurait maintenant 16 points sur 40, soit la moitié correspondant à la proportion des points obtenus pour les fonctionnalités).
3. Dans le cas d'un retard, les règles de l'article 6.1.11 de la PÉA seront appliquées : *« les rapports et les travaux exigés des étudiants et des étudiantes doivent être remis au professeur ou à la professeure à la date, à l'heure et au lieu indiqués. Une journée ouvrable de retard est tolérée entraînant une pénalité de 15 % de la note. Au-delà de ce délai, le travail est refusé et la note « 0 » est attribuée. »*

6 Modalités de remise

Remettez votre projet sur LÉA, dans la section Travaux, à l'intérieur d'une archive *Zip*. Supprimez **tous les fichiers et dossiers temporaires**. Vous pouvez également remettre votre diagramme de classe si vous l'avez complété. Faites une seule remise par équipe (mettez le nom de votre coéquipier en commentaire lors de la remise).